WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

仓颉鸿蒙实战之复刻豆包UI



主讲人：徐迪

01 项目相关

02 UI界面

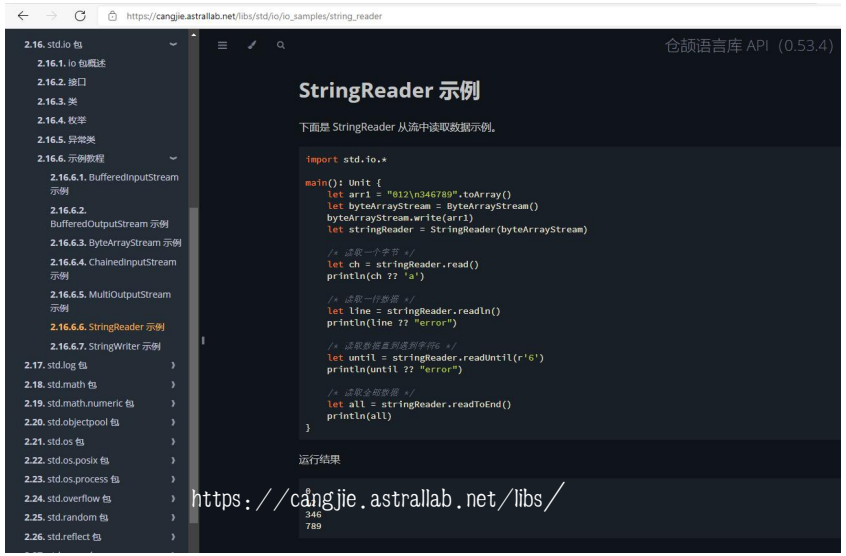
03 调用语言大模型API

04 上下文与流式输出

WORKSHOP 研讨/分享/感悟

学习参考

仓颉语言库 API



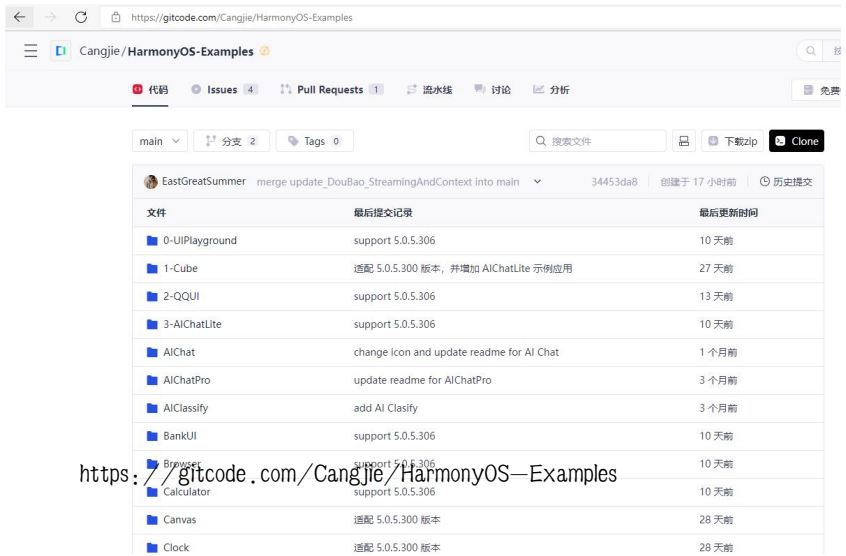
https://cangjie.astrallab.net/libs/

HarmonyOS — 仓颉 API 参考



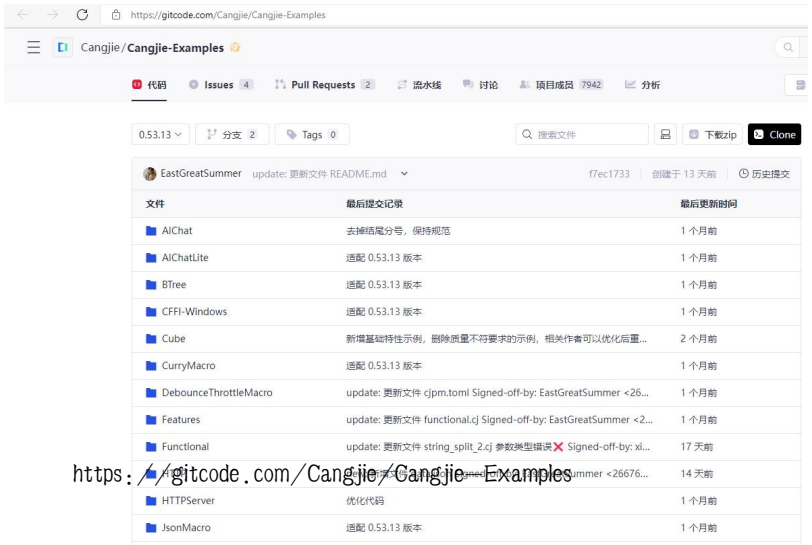
https://developer.huawei.com/consumer/cn/doc/cangjie-references-V5/ui_u7ec4_u4ef6_uff08_u4ed3_u9889_uff09-V5

仓颉鸿蒙应用示例代码仓库



https://gitcode.com/Cangjie/HarmonyOS-Examples

仓颉示例代码仓库



https://gitcode.com/Cangjie/Cangjie-Examples

仓颉 Cangjie

WORKSHOP 研讨/分享/感悟

页面展示 对话页



仓颉 Cangjie

WORKSHOP 研讨/分享/感悟

页面展示 创作页



仓颉 Cangjie

WORKSHOP 研讨/分享/感悟

页面展示 发现页



仓颉 Cangjie

页面展示 通知页


05:04
100
通知
写作·生图·读长文
就用豆包电脑版
doubao.com
写作
图像生成
AI 阅读
暂未收到通知
对话
发现
创作
通知
我的

页面展示 个人页


09:36
100
Admin
豆包号:pc66336
编辑个人资料
作品
私密
智能体
收藏
你发布的图片、音乐将展示在这里
去创作
对话
发现
创作
通知
我的

UI拆解示例

Swiper+Tab

List

Tab

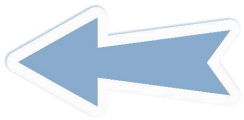



```
ListItem：
Column()
    Row()
        Image()
        Column()
            Row()
                Row()
                    Text()
                    ...
```

WORKSHOP 研讨/分享/感悟

调用语言大模型 API

```
let info = """
{
  "model": "deepseek-chat",
  "messages":
  [
    {
      "role": "system",
      "content": "你是我的全能助手，你叫仓宝"
    },
    {
      "role": "user",
      "content": "${question}“
    }
  ],
  "stream": false
}
"""
```



Request

APPLICATION/JSON

BODY REQUIRED

messages object[] REQUIRED

Possible values: >= 1

对话的消息列表。

Array [

oneOf

System message | User message | Assistant message | Tool message

content string REQUIRED

system 消息的内容。

role string REQUIRED

Possible values: [system]

该消息的发起角色，其值为 system。

name string

可以选填的参与者的名称，为模型提供信息以区分相同角色的参与者。

]

model string REQUIRED

Possible values: [deepseek-chat, deepseek-reasoner]

使用的模型的 ID。您可以使用 deepseek-chat。

stream boolean NULLABLE

如果设置为 True，将会以 SSE（server-sent events）的形式以流式发送消息增量。消息流以 data: [DONE] 结尾。

仓颉 Cangjie

调用语言大模型 API

```
func parse(text: String) {
    let json = JsonValue.fromStr(text).asObject()
    let choices = json.getFields()['choices'].asArray()
    let message = choices[0].asObject()
                        .getFields()['message'].asObject()
    let content = message.getFields()['content']
                        .asString().getValue()
    return content
}
```



Responses

200 (No streaming) 200 (Streaming)

OK, 返回一个 `chat completion` 对象。

APPLICATION/JSON

Schema Example (from schema) Example

SCHEMA

id string REQUIRED
该对话的唯一标识符。

choices object[] REQUIRED
模型生成的 completion 的选择列表。

Array [

finish_reason string REQUIRED
Possible values: [`stop`, `length`, `content_filter`, `tool_calls`, `insufficient_system_resource`]
模型停止生成 token 的原因。
`stop`：模型自然停止生成，或遇到 `stop` 序列中列出的字符串。
`length`：输出长度达到了模型上下文长度限制，或达到了 `max_tokens` 的限制。
`content_filter`：输出内容因触发过滤策略而被过滤。
`insufficient_system_resource`：系统推理资源不足，生成被打断。

index integer REQUIRED
该 completion 在模型生成的 completion 的选择列表中的索引。

message object REQUIRED
模型生成的 completion 消息。

content string NULLABLE REQUIRED
该 completion 的内容。

上下文拼接

DeepSeek /chat/completions API 是一个“无状态” API，即服务端不记录用户请求的上下文，用户在每次请求时，**需将之前所有对话历史拼接好后**，传递给对话 API。

```
@State var messages: StringBuilder =  StringBuilder("...")
 ...
messages.append(histroy) //将历史对话添加到 messages 末尾
messages.append(question) //将每轮用户请求添加到 messages 末尾
messages.append(answer) //将每轮模型的输出添加到 messages 末尾
```

最终传递给 API 的 `messages` 为：

```
[
    {"role": "user", "content": "What's the highest mountain in the world?"},
    {"role": "assistant", "content": "The highest mountain in the world is Mount Everest."},
    {"role": "user", "content": "What is the second?"}
]
```

流式对话

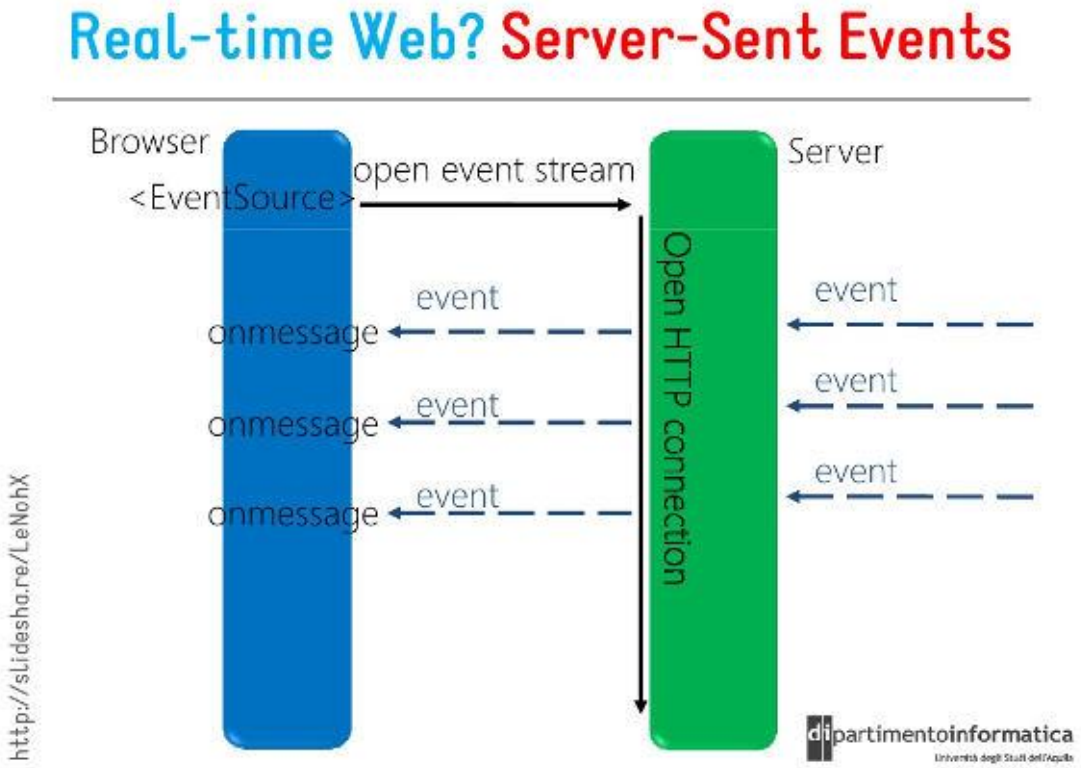
流式 HTTP 通信原理

1. SSE 的本质

严格地说，HTTP 协议无法做到服务器主动推送信息。但是，有一种变通方法，就是服务器向客户端声明，接下来要发送的是流信息（streaming）。

也就是说，发送的不是一次性的数据包，而是一个数据流，会连续不断地发送过来。这时，客户端不会关闭连接，会一直等着服务器发过来的新的数据流，视频播放就是这样的例子。本质上，这种通信就是以流信息的方式，完成一次用时很长的下载。

SSE 就是利用这种机制，使用流信息向浏览器推送信息。它基于 HTTP 协议，目前除了 IE/Edge，其他浏览器都支持。



流式对话

流式 HTTP 通信原理

2. 基本用法

```
let info = """
{
  "model": "deepseek-chat",
  "messages":
  [
    {
      "role": "system",
      "content": "你是我的全能助手，你叫仓宝"
    }
  ],
  "stream": true
}
"""
```

```
let request = HttpRequestBuilder()
  .url('https://api.deepseek.com/chat/completions')
  .header('Authorization', 'Bearer ${DEEPSEEK_API_KEY}')
  .header('Content-Type', 'application/json')
  .header("Accept", "text/event-stream")
  .body(info)
  .readTimeout(Duration.second * 5)
  .post()
  .build()
var response = this.send(request)
...
while(tempText != "[DONE]") {
  let buffer = Array<Byte>(10240, item: 0)
  let length = response.body.read(buffer)
  let rawText = String.fromUtf8(buffer[..length])
  for (tempText in rawText.split("\n")){
    resText.append(parseContent(tempText))
  }
...
}
```

最后以 [DONE] 结尾

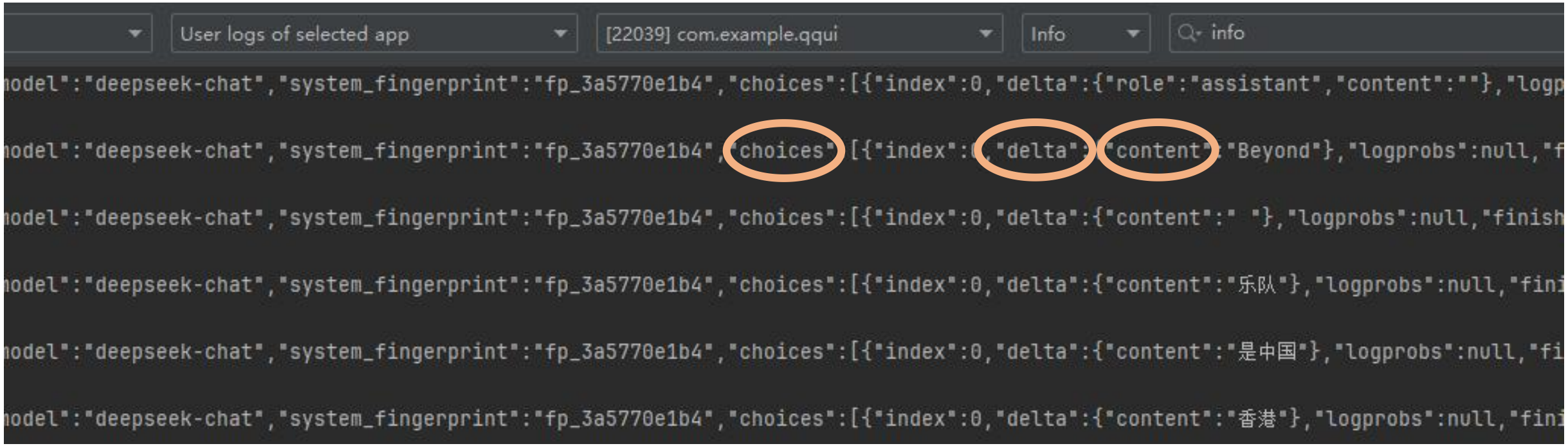
仓颉 Cangjie

流式对话

流式 HTTP 通信原理

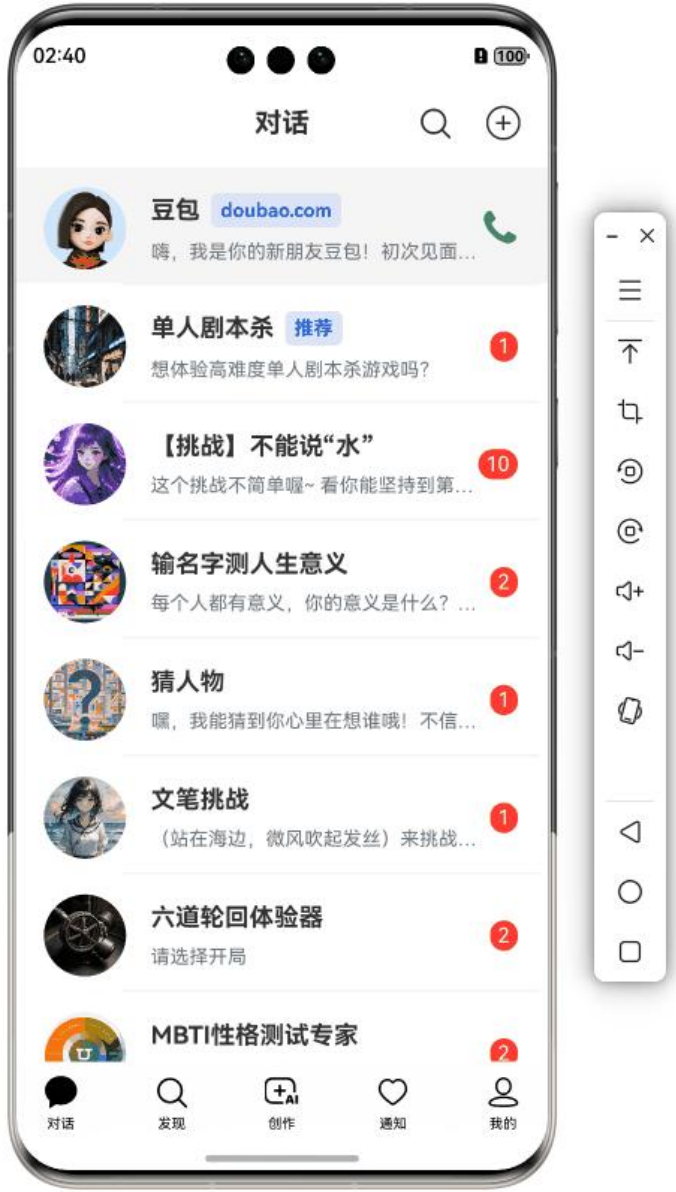
2. 基本用法

```
func parseContent(text: String) {
  let json = JsonValue.fromStr(text).asObject()
  let choices = json.getFields()['choices'].asArray()
  let message = choices[0].asObject().getFields()['delta'].asObject()
  let content = message.getFields()['content'].asString().getValue()
  return content
}
```



WORKSHOP 研讨/分享/感悟

功能演示



AIChat支持上下文与流式对话

仓颉 Cangjie

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，
获取最新官方资讯!

